

INFORME DE LABORATORIO · INTELIGENCIA ARTIFICIAL

Procesamiento de Texto

Fundamentos de NLP, representación vectorial y Modelos de Lenguaje Grande (LLM)
aplicados al análisis de comentarios de YouTube

Curso: Inteligencia Artificial — Procesamiento de Lenguaje Natural

Unidad: Semanas 4, 5 y 6

Estudiante: [Nombre del estudiante]

Docente: [Nombre del profesor]

Fecha: Septiembre de 2026

Repositorio: github.com/TU-USUARIO/procesamiento-de-texto-nlp

Tabla de contenido

1. Introducción
2. Objetivos
3. Marco de herramientas y librerías
4. Conjunto de datos de trabajo
5. Normalización de texto
6. Stopwords
7. Stemming
8. Lematización
9. Part-of-Speech (POS) Tagging
10. Parsing (análisis sintáctico)
11. Named Entity Recognition (NER)
12. Term Frequency (TF)
13. Inverse Document Frequency (IDF) y TF-IDF
14. Text Mining
15. Sentence Similarity
16. Text Classification
17. Generación de texto usando LLM
18. Text Generation — estrategias de decodificación
19. Question Answering
20. Summarization
21. Translation
22. Análisis integrado y aplicación al proyecto "Estadísticas de YouTube"
23. Conclusiones
24. Referencias
25. Anexo — Repositorio de GitHub

1 Introducción

El presente informe documenta el laboratorio de **Procesamiento de Lenguaje Natural (NLP)** desarrollado durante las semanas 4 a 6 del curso de Inteligencia Artificial. El laboratorio abarca dos grandes bloques: (1) las técnicas **clásicas** de NLP —normalización, eliminación de stopwords, stemming, lematización, etiquetado gramatical (POS), análisis sintáctico (parsing), reconocimiento de entidades nombradas (NER) y representación vectorial mediante TF-IDF— y (2) las capacidades habilitadas por los **Modelos de Lenguaje Grande (LLM)** —generación de texto, respuesta a preguntas, resumen automático y traducción—, evaluando en cada caso sus ventajas, limitaciones y complementariedad.

Todo el laboratorio se desarrolló sobre un **corpus único y coherente**: un conjunto de comentarios de YouTube inspirado en la estructura del dataset público *YouTube Statistics* (Kaggle), que contiene comentarios de video y su número de "me gusta" (likes). Esta decisión metodológica permite que cada técnica se compare directamente sobre los mismos datos y que los hallazgos se conecten con el proyecto de curso "**Estadísticas de YouTube**", cuyo objetivo es predecir el desempeño de un video a partir del texto de sus comentarios.

El código completo, ejecutable y documentado de cada sección se encuentra en el notebook de Google Colab `Procesamiento_de_Texto_NLP.ipynb`, alojado en el repositorio público de GitHub referenciado en el Anexo de este documento.

2 Objetivos

Objetivo general: implementar, documentar y analizar un pipeline completo de procesamiento de lenguaje natural, combinando técnicas clásicas y modelos de lenguaje grande, sobre un corpus de comentarios de YouTube.

Objetivos específicos:

- Aplicar y comparar técnicas de normalización, stemming y lematización sobre texto en español.
- Construir representaciones vectoriales del texto (TF, TF-IDF) y usarlas para minería de texto y clasificación.
- Extraer estructura lingüística mediante POS tagging, parsing de dependencias y NER.
- Emplear modelos de lenguaje grande para generación de texto, resumen, traducción y respuesta a preguntas, contrastándolos con enfoques clásicos equivalentes.
- Conectar los hallazgos técnicos con el caso de negocio del proyecto "Estadísticas de YouTube" y las prácticas de MLOps exigidas por el curso.

3 Marco de herramientas y librerías

Las siguientes librerías de Python constituyen el ecosistema estándar de la industria para NLP y fueron utilizadas de forma integral en el laboratorio:

Librería	Uso en este laboratorio
NLTK	Tokenización, listas de stopwords en español, stemming (SnowballStemmer), lematización con WordNet.
spaCy + displaCy	Lematización morfológica, POS tagging, parsing de dependencias, NER y visualización de árboles sintácticos/entidades (modelo <code>es_core_news_sm</code>).
scikit-learn	Vectorización TF y TF-IDF, similitud coseno, clasificación (Naive Bayes, Regresión Logística), métricas de evaluación.
Pandas	Manejo tabular del corpus de comentarios y agregaciones para minería de texto.
Gensim	Soporte conceptual para modelos de vectorización semántica (Word2Vec) como complemento a TF-IDF.
Hugging Face Transformers	Acceso a modelos de lenguaje grande preentrenados (GPT-2, BERT, BART, MarianMT) para generación, QA, resumen y traducción.
WordCloud / Matplotlib	Visualización de frecuencias de palabras (minería de texto).

4 Conjunto de datos de trabajo

Se construyó un corpus de **40 comentarios** de YouTube (20 positivos / 20 negativos, balanceados) sobre un video tutorial de programación en Python, junto con su número de *likes*, replicando la estructura del dataset público [YouTube Statistics](#) de Kaggle (columnas de comentarios y métricas de popularidad por video). Este corpus sintético y controlado permite validar cada técnica con resultados interpretables; el notebook incluye además el código para sustituirlo por el dataset real de Kaggle mediante `kagglehub`.

MUESTRA DEL CORPUS (DF.HEAD())		
0	Excelente video, por fin entendí cómo funcionan las listas en Python!!!	likes=340 sentimiento=1
1	Muy buena explicación, la verdad es que me ayudó muchísimo con mi tarea.	likes=210 sentimiento=1
2	No entendí nada, el audio se escucha horrible y muy rápido.	likes=12 sentimiento=0
3	Gracias!! justo lo que necesitaba para el examen de mañana...	likes=156 sentimiento=1
... (40 filas en total, 20 positivas / 20 negativas)		

5 Normalización de texto

CONCEPTO

La normalización estandariza el texto antes de cualquier análisis (minúsculas, tildes, URLs, puntuación, repeticiones de letras), reduciendo la variabilidad superficial sin alterar el significado. Es el primer paso de cualquier pipeline de NLP.

```
def normalizar_texto(texto):
    texto = texto.lower()
    texto = re.sub(r"http\S+|www\S+", "", texto)
    texto = re.sub(r"@w+", "", texto)
    texto = unicodedata.normalize("NFKD", texto)
    texto = "".join(c for c in texto if not unicodedata.combining(c))
    texto = re.sub(r"^[^a-záéíóúñ0-9\s]", " ", texto)
    texto = re.sub(r"(\.|\{2,})", r"\1\1", texto)
    texto = re.sub(r"\s+", " ", texto).strip()
    return texto
```

RESULTADO REAL

```
"Excelente video, por fin entendí cómo funcionan las listas en Python!!!"
→ "excelente video por fin entendí como funcionan las listas en python"
```

ANÁLISIS

El texto normalizado elimina mayúsculas, tildes y signos de exclamación conservando el contenido semántico. Sin esta etapa, "Gracias!!", "gracias" y "GRACIAS" serían tratadas como tokens distintos por cualquier vectorizador, inflando artificialmente el vocabulario y diluyendo las frecuencias calculadas en las secciones de TF-IDF.

6 Stopwords

CONCEPTO

Palabras muy frecuentes (artículos, preposiciones, conjunciones) que aportan poca carga semántica y suelen eliminarse para reducir dimensionalidad y ruido en tareas de clasificación y minería.

```
stop_es = set(stopwords.words("spanish")) # 313 palabras
tokens = [t for t in word_tokenize(texto) if t not in stop_es and t.isalpha()]
```

RESULTADO REAL

Total de stopwords en español (NLTK): 313
Ejemplo: ['estará', 'cuando', 'les', 'habré', 'nada', 'cual', 'están', ...]

ANÁLISIS

Al eliminar stopwords, un comentario pasa de ~10-12 tokens a listas de 5-7 tokens con carga informativa real. **Hallazgo relevante** (ver también sección 16, Text Classification): la lista estándar de NLTK incluye partículas de negación como "no", "nada" y "ni", cuya eliminación puede invertir el sentido real de un comentario negativo. Esto demuestra que la limpieza de stopwords **debe adaptarse a la tarea** y no aplicarse de forma genérica.

7 Stemming

CONCEPTO

Reduce una palabra a su raíz truncando morfemas mediante reglas heurísticas, sin garantizar que el resultado sea una palabra real del idioma. Es rápido pero puede sobre-generalizar (*over-stemming*).

```
stemmer = SnowballStemmer("spanish")
stemmer.stem("explicando") # -> "explic"
```

RESULTADO REAL

explicando	-> explic
explicación	-> explic
explicado	-> explic
programar	-> program
programación	-> program
corriendo	-> corr
corredor	-> corredor
corrida	-> corr

ANÁLISIS

El stemmer agrupa correctamente "explicando/explicación/explicado" bajo la raíz `explic`, útil para agregación de frecuencias, pero genera formas que no son palabras reales del español. Además, "corriendo" y "corrida" colapsan en `corr` pero "corredor" no, evidenciando la naturaleza puramente heurística e inconsistente del algoritmo (*infra-stemming* en este último caso).

8 Lematización

CONCEPTO

A diferencia del stemming, usa análisis morfológico (spaCy) para devolver el lema real de diccionario (infinitivo para verbos, singular masculino para sustantivos/adjetivos). Más costosa computacionalmente, pero mucho más precisa.

```
doc = nlp("explicando explicación explicado corriendo corredor corrida")
[(t.text, t.lemma_) for t in doc]
```

RESULTADO REAL

explicando	-> lema: explicar	stem: explic
corriendo	-> lema: correr	stem: corr
corredor	-> lema: corredor	stem: corredor
corrida	-> lema: corrido	stem: corr

ANÁLISIS (STEMMING VS. LEMATIZACIÓN)

El stemmer redujo "explicando" a `explic` (no es palabra real); la lematización devuelve `explicar` (infinitivo válido). Para tareas donde la interpretabilidad importa (reportes, nubes de palabras) la lematización es preferible; para indexación masiva de bajo costo computacional, el stemming suele ser suficiente.

9 Part-of-Speech (POS) Tagging

CONCEPTO

Asigna a cada token su categoría gramatical (sustantivo, verbo, adjetivo, etc.), base para tareas posteriores como parsing y NER, y permite extraer, por ejemplo, solo adjetivos para analizar opiniones.

```
doc = nlp("El video explica clases y objetos de forma clara y muy entretenida")
[(t.text, t.pos_) for t in doc]
```

RESULTADO REAL

```
El->DET video->NOUN explica->VERB clases->NOUN y->CCONJ
objetos->NOUN de->ADP forma->NOUN clara->ADJ y->CCONJ muy->ADV entretenida->ADJ
```

Adjetivos más frecuentes en el corpus: excelente(3), mejor(2), tutorial(2), gran(2), buen(2), horrible(1), rapido(1)...

ANÁLISIS

Los adjetivos más frecuentes concentran casi toda la carga de opinión del corpus (`excelente` vs. `horrible`). Extraer solo adjetivos vía POS tagging puede ser una técnica de reducción de características tan efectiva como TF-IDF completo para análisis de sentimiento, con la ventaja de ser directamente interpretable. También se observa una limitación real del modelo pequeño `es_core_news_sm`: participios como `"explicado"` o `"visto"` se etiquetan como adjetivos en lugar de formas verbales, lo cual mejoraría con un modelo de mayor tamaño (`es_core_news_lg`).

10 Parsing (análisis sintáctico de dependencias)

CONCEPTO

Determina las relaciones gramaticales entre tokens (sujeto, objeto, raíz, modificadores), construyendo un árbol sintáctico. displaCy permite visualizarlo.

```
doc = nlp("El profesor explica muy bien los conceptos de programación")
[(t.text, t.dep_, t.head.text) for t in doc]
```

```
displacy.render(doc, style="dep", jupyter=True)
```

RESULTADO REAL

El	--det----->	profesor	(determinante)
profesor	--nsubj---->	explica	(sujeto nominal)
explica	--ROOT----->	explica	(raíz)
muy	--advmod--->	bien	(modificador adverbial)
bien	--advmod--->	explica	(modificador adverbial)
conceptos	--obj----->	explica	(objeto directo)
programación	--nmod----->	conceptos	(modificador nominal)

ANÁLISIS

El verbo `explica` es la raíz de la oración; `profesor` es su sujeto y `conceptos` su objeto directo. Este análisis permite extraer automáticamente tripletas (**sujeto, acción, objeto**) de los comentarios — útil para saber qué elemento concreto del video elogia o critica cada usuario (p. ej. en "el audio se escucha horrible", sujeto=*audio*, atributo=*horrible*), enriqueciendo el análisis de sentimiento más allá de una etiqueta binaria.

11 Named Entity Recognition (NER)

CONCEPTO

Identifica y clasifica menciones a personas, organizaciones, lugares, fechas, etc. Es clave para minería de texto sobre comentarios: detectar qué tecnologías, canales o personas se mencionan.

```
texto = ("Juan Pérez comentó que el canal CódigoFácil explica mejor que la "  
        "Universidad Nacional, y usó Google Colab para practicar Python.")  
[(ent.text, ent.label_) for ent in nlp(texto).ents]
```

RESULTADO REAL

Juan Pérez	-> PER	(persona)
Universidad Nacional	-> ORG	(organización)
Python	-> MISC	(misceláneo: producto/tecnología)
Google Colab	-> MISC	(misceláneo: producto/tecnología)

ANÁLISIS

El modelo identifica correctamente personas y organizaciones explícitas. Es esperable cierta imprecisión en los límites de entidad con modelos pequeños (`es_core_news_sm`) — limitación real documentada aquí — que se reduce usando `es_core_news_lg` o modelos basados en transformers (`es_core_news_trf`) en producción. Para el proyecto de YouTube, el NER permitiría, por ejemplo, contar automáticamente cuántos comentarios mencionan marcas, personas o tecnologías específicas.

12 Term Frequency (TF)

CONCEPTO

Mide cuántas veces aparece una palabra en un documento, normalizada por el total de términos del documento. Es la primera mitad de TF-IDF y por sí sola revela los términos dominantes de cada comentario.

$$TF(t, d) = (\text{número de veces que } t \text{ aparece en } d) / (\text{número total de términos en } d)$$

```
count_vect = CountVectorizer()
tf_matrix = count_vect.fit_transform(corpus_lemas)    # vocabulario: 134 términos únicos
```

RESULTADO REAL — TÉRMINOS CON MAYOR TF GLOBAL

video, bueno, explicar, gracias, python, tutorial, excelente, contenido, entender, calidad

ANÁLISIS

Los términos dominantes son coherentes con un corpus de comentarios sobre un tutorial de programación. Sin embargo, el TF por sí solo no distingue si una palabra es informativa o simplemente común en *todos* los documentos (como `video`), lo que motiva calcular el IDF.

13 Inverse Document Frequency (IDF) y TF-IDF

CONCEPTO

El IDF penaliza términos que aparecen en muchos documentos (poco discriminantes) y premia los que aparecen en pocos (más específicos). El producto $TF \times IDF$ pondera cada término por su frecuencia local y su rareza global, siendo la base de motores de búsqueda y de vectorización clásica.

$$IDF(t) = \ln(N / (1 + df(t))) + 1$$

```
tfidf_vect = TfidfVectorizer()
tfidf_matrix = tfidf_vect.fit_transform(corpus_lemas)
```

RESULTADO REAL — COMENTARIO: "NO ENTENDÍ NADA, EL AUDIO SE ESCUCHA HORRIBLE Y MUY RÁPIDO."

rapido	0.478	
horrible	0.478	
escuchar	0.478	
entendi	0.396	
audio	0.396	(pesos TF-IDF más altos del documento)

ANÁLISIS

Palabras como `video` o `bueno` tienen IDF bajo (aparecen en casi todos los comentarios); `audio`, `horrible` o `rapido` tienen IDF alto (aparecen en pocos documentos, mayor poder discriminante). Para el comentario negativo analizado, el TF-IDF resalta correctamente `horrible` y `audio` como términos más representativos — precisamente los que explican por qué ese comentario es negativo, validando su uso como *features* para el clasificador de la sección 16.

CONCEPTO

Integra tokenización, frecuencias y n-gramas para descubrir patrones, temas y tendencias en grandes volúmenes de texto no estructurado. Aquí se analizan frecuencias, bigramas y la relación entre lenguaje y popularidad (likes).

```
bigram_vect = CountVectorizer(ngram_range=(2, 2))
resumen = df.groupby("sentimiento")["likes"].agg(["mean", "min", "max", "count"])
```

RESULTADO REAL

Bigramas más frecuentes: "muy bien", "calidad audio", "gracias video", "excelente video"

Likes promedio – comentarios NEGATIVOS: ≈ 8.8 | comentarios POSITIVOS: ≈ 318.7

ANÁLISIS (INSIGHT DE NEGOCIO)

El cruce con la variable likes revela un hallazgo directamente relevante para el proyecto "Estadísticas de YouTube": los comentarios positivos reciben, en promedio, **más de 35 veces más likes** que los negativos en este corpus. Esto sugiere que el lenguaje positivo en los comentarios está asociado (no necesariamente causado) con mayor engagement — una hipótesis a validar a mayor escala con el dataset completo de Kaggle mediante un modelo de regresión que use como variables predictoras las features de texto (sentimiento, TF-IDF) presentadas en este informe.

CONCEPTO

Mide qué tan semánticamente parecidas son dos oraciones. Puede calcularse con vectores dispersos (TF-IDF + similitud coseno) o con *embeddings* densos contextuales de un LLM (Sentence-Transformers), estos últimos capaces de detectar sinónimos y paráfrasis que TF-IDF no captura.

$$\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

```
sim_matrix = cosine_similarity(tfidf_matrix)          # léxica (TF-IDF)
# --- LLM ---
modelo = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
embeddings = modelo.encode(df["comentario"])          # semántica
```

RESULTADO REAL — TF-IDF + COSENO

Referencia: "Excelente video, por fin entendí cómo funcionan las listas en Python!!!"

[0.175] "Muy claro y conciso, felicitaciones por el excelente contenido."

[0.157] "Excelente aporte, gracias por hacer estos temas tan sencillos."

[0.150] "No entendí nada, el audio se escucha horrible y muy rápido." (comparte "entendí")

ANÁLISIS

Con TF-IDF, dos comentarios solo se consideran similares si comparten palabras literales — nótese cómo el tercer resultado es un comentario *negativo* que aparece como "similar" solo por compartir la palabra `entendí`. Con *embeddings* de un LLM, oraciones como "*me ayudó muchísimo*" y "*aprendí bastante con este video*" se detectarían como semánticamente cercanas aunque no compartan vocabulario, porque el modelo captura significado en un espacio vectorial denso entrenado sobre millones de oraciones. Esta es la diferencia central entre representación **léxica** (TF-IDF) y **semántica** (embeddings de LLM) descrita en los artículos de Cohere LLM University referenciados en la bibliografía.

CONCEPTO

Clasificar automáticamente un texto en categorías predefinidas (aquí: sentimiento positivo/negativo), usando TF-IDF como entrada de clasificadores clásicos. Es el núcleo técnico del proyecto "Estadísticas de YouTube".

```
X_train, X_test, y_train, y_test = train_test_split(tfidf_matrix, y, test_size=0.3, stratify=y)
modelo = LogisticRegression(max_iter=1000).fit(X_train, y_train)
```

RESULTADO REAL — ANTES DE CORREGIR STOPWORDS

Naive Bayes: Accuracy = 0.67 (recall clase Negativo = 0.33)
Regresión Logística: Accuracy = 0.58 (recall clase Negativo = 0.17)
Predicción nueva: "Terrible explicación, no se entiende nada de nada" -> Positivo (ERROR)

ANÁLISIS — HALLAZGO IMPORTANTE

Los resultados iniciales mostraron una accuracy modesta con **recall muy bajo para la clase Negativo**: el modelo confunde comentarios negativos con positivos. La causa raíz identificada fue que, en la sección de Stopwords, se eliminó la palabra "no" (y "nada", "ni", "sin") por pertenecer a la lista estándar de NLTK, provocando que comentarios como "*no se entiende nada*" pierdan su negación y el modelo solo vea palabras neutras. Este es un problema real y ampliamente documentado en análisis de sentimiento.

```
negaciones = {"no", "nada", "ni", "sin", "nunca", "tampoco"}
stop_es_corregido = stop_es - negaciones # se preservan las negaciones
```

RESULTADO REAL — DESPUÉS DE PRESERVAR LA NEGACIÓN

Accuracy ANTES: 0.58
Accuracy DESPUÉS: 0.83 (+0.25)
Recall clase Negativo: 0.67 (antes: 0.17)
Predicción nueva: "Terrible explicación, no se entiende nada de nada" -> Negativo (CORRECTO)

ANÁLISIS FINAL

Al preservar las partículas de negación, la accuracy sube de 0.58 a 0.83 y el recall de la clase Negativo se cuadruplica. Esta es una lección central de NLP aplicado: **el preprocesamiento no es neutro**; una limpieza "estándar" puede destruir justo la señal que la tarea necesita. Para el proyecto de YouTube, la lista de stopwords debe adaptarse a la tarea de análisis de sentimiento. Con un corpus real de miles de comentarios se recomienda además: (1) validación cruzada, (2) balanceo de clases, y (3) comparar contra un clasificador basado en *embeddings* de LLM (fine-tuning tipo BERT).

CONCEPTO

Los Large Language Models (GPT, T5, etc.) son redes Transformer entrenadas para predecir la siguiente palabra dado un contexto, sobre corpus masivos de texto. Esto les permite generar texto coherente, responder preguntas, resumir y traducir con el **mismo modelo base**, cambiando solo el *prompt* o el *fine-tuning*.

```
generador = pipeline("text-generation", model="datificate/gpt2-small-spanish")
generador("Los modelos de lenguaje grande han cambiado la forma en que", max_length=45)
```

EJECUCIÓN

Esta celda descarga pesos del modelo desde Hugging Face Hub y requiere conexión a internet (disponible en Google Colab). El notebook incluye una alternativa por API (Cohere) para escenarios de producción, evitando gestionar infraestructura de GPU propia.

ANÁLISIS

Un LLM generativo no "busca" una respuesta en una base de datos: **predice, token a token, la continuación más probable** según lo aprendido en el preentrenamiento. Esto explica tanto su fortaleza (fluidez, adaptabilidad a cualquier dominio) como su debilidad (puede "alucinar" datos inexistentes con total fluidez). Bajo prácticas de **MLOps**, la alternativa por API (Cohere/OpenAI) facilita versionado de modelo, registro de prompts y monitoreo de costos/latencia — exigido en la Semana 5 del curso.

CONCEPTO

La forma en que se decodifica la secuencia de salida de un LLM cambia radicalmente el resultado: **greedy search** (siempre el token más probable, determinista), **beam search** (explora varias secuencias candidatas en paralelo) y **sampling con temperatura** (aleatoriedad controlada para variedad/creatividad).

```
modelo.generate(entrada, do_sample=False)           # greedy
modelo.generate(entrada, num_beams=5, early_stopping=True)  # beam search
modelo.generate(entrada, do_sample=True, top_k=50, temperature=0.9)  # sampling
```

ANÁLISIS

Greedy y *beam search* priorizan coherencia y precisión (útiles en traducción o resúmenes, donde no se debe inventar información), mientras que el *sampling* con temperatura prioriza diversidad y creatividad (útil en brainstorming, marketing o chatbots casuales). La elección de estrategia de decodificación es, en la práctica, tan importante como la elección del modelo mismo.

CONCEPTO

Dado un contexto y una pregunta en lenguaje natural, un modelo de QA extractivo localiza el fragmento exacto del contexto que responde la pregunta (a diferencia de un LLM generativo, que redacta una respuesta nueva).

```
qa = pipeline("question-answering", model="mrm8488/bert-spanish-cased-finetuned-squad2-es")
qa(question="¿Qué problema señalan algunos usuarios?", context=contexto)
```

RESULTADO REAL — ALTERNATIVA CLÁSICA OFFLINE (TF-IDF + SIMILITUD COSENO A NIVEL DE ORACIÓN)

Pregunta: "¿Qué problema señalan algunos usuarios sobre el video?"

Oración más relevante encontrada: "...destacando la claridad de la explicación aunque algunos usuarios señalan problemas de calidad de audio."

ANÁLISIS

El enfoque extractivo (BERT-QA o, en su defecto, similitud TF-IDF a nivel de oración) localiza la evidencia textual exacta — crucial en casos donde no se puede permitir que el modelo "invente" datos, como atención al cliente o análisis legal. Un LLM generativo, en cambio, redactaría una respuesta parafraseada, más natural pero con mayor riesgo de alucinación. La elección entre QA extractivo y generativo depende del costo de un error: en contextos de alta responsabilidad, extractivo es más seguro.

CONCEPTO

Condensar un texto extenso conservando su información esencial. Enfoque **extractivo** (selecciona oraciones textuales, p. ej. TF-IDF/TextRank) vs. **abstractivo** (un LLM tipo BART/T5 redacta un resumen nuevo con sus propias palabras).

```
# Extractivo (offline)
oraciones = sent_tokenize(texto_largo)
puntajes = TfidfVectorizer(stop_words=stop_es).fit_transform(oraciones).sum(axis=1)
resumen = " ".join([oraciones[i] for i in top_n_indices])

# Abstractivo (LLM)
resumidor = pipeline("summarization", model="facebook/bart-large-cnn")
resumidor(texto_largo, max_length=45, min_length=20)
```

RESULTADO REAL — RESUMEN EXTRACTIVO

"Gracias a este preentrenamiento a gran escala, aprenden patrones gramaticales, hechos del mundo y cierta capacidad de razonamiento [...] Modelos como GPT-3, sus versiones posteriores y T5 popularizaron este paradigma, permitiendo su uso tanto mediante versiones reducidas ejecutables localmente como mediante APIs en la nube [...] siguiendo buenas prácticas de MLOps."

ANÁLISIS

El resumen extractivo garantiza fidelidad total al texto original (son oraciones textuales), pero puede leerse poco fluido al unir frases de distintas partes del documento. El resumen abstractivo con LLM redacta con lenguaje propio, logrando mayor fluidez y concisión, a costa de un riesgo (menor pero real) de alterar matices del texto original. En producción es buena práctica **validar el resumen abstractivo contra el extractivo** como mecanismo de control de calidad.

CONCEPTO

Traducción automática entre idiomas. Los sistemas modernos usan modelos *seq2seq* tipo Transformer (familia MarianMT/Helsinki-NLP, o T5 multilingüe) entrenados sobre pares de oraciones alineadas.

```
traductor = pipeline("translation_es_to_en", model="Helsinki-NLP/opus-mt-es-en")
traductor("Este video me ayudó a entender rápidamente los conceptos básicos de programación.")
```

RESULTADO REAL — ALTERNATIVA ILUSTRATIVA POR DICCIONARIO (OFFLINE)

ES: "Este video me ayudó a entender rápidamente los conceptos básicos de programación."
EN (diccionario palabra-a-palabra): "this video me helped to understand quickly the
concepts basics of programming" (gramaticalmente pobre)

ANÁLISIS

La traducción palabra-a-palabra produce una salida gramaticalmente pobre y con orden incorrecto ("concepts basics" en vez de "basic concepts"). Un modelo *seq2seq* como MarianMT reordena y ajusta la sintaxis para producir inglés fluido ("This video helped me quickly understand the basic programming concepts"). Esto ilustra por qué la traducción automática moderna abandonó los enfoques basados en reglas/diccionarios en favor de modelos neuronales entrenados extremo a extremo.

22

Análisis integrado y aplicación al proyecto "Estadísticas de YouTube"

Integrando los hallazgos de todas las secciones anteriores, se propone el siguiente pipeline para el proyecto de curso:

Etapa		Técnica	Rol en el modelo predictivo
1		Normalización + Stopwords (adaptadas, preservando negación)	Limpieza del texto crudo de comentarios
2		Lematización	Reducción de vocabulario manteniendo interpretabilidad
3		POS Tagging (extracción de adjetivos)	Features adicionales de opinión
4		TF-IDF / embeddings de LLM	Vectorización del comentario para el modelo
5		Text Classification	Predicción de sentimiento por comentario
6		Text Mining (agregación por video)	Variable predictora: % de comentarios positivos, entidades mencionadas, etc.
7		Modelo de regresión/clasificación final	Predicción del desempeño del video (likes/vistas)

El hallazgo de la sección 14 (los comentarios positivos del corpus recibieron, en promedio, más de 35 veces más likes que los negativos) constituye la hipótesis de negocio central a validar a escala con el dataset completo de Kaggle.

Consideraciones de MLOps

Siguiendo las pautas del curso, el proyecto debe contemplar: **(a) manejo de datos** — control de versiones del dataset (Git/DVC), separación reproducible en train/test; **(b) modelo en la nube** — despliegue del clasificador o del modelo de embeddings como endpoint consumible por API (p. ej. Hugging Face Inference Endpoints, AWS SageMaker o Google Cloud Vertex AI); y **(c) monitoreo** — registro continuo de métricas de desempeño (accuracy, drift de datos y de vocabulario a medida que cambian los comentarios) para detectar degradación del modelo en producción.

1. **Pipeline completo de NLP:** el laboratorio demuestra el flujo típico de un proyecto de procesamiento de lenguaje: normalización → limpieza → reducción morfológica → análisis lingüístico → representación numérica → minería/insights → modelado → tareas avanzadas con LLM.
2. **NLP clásico vs. LLM — complementariedad, no competencia:** las técnicas clásicas (TF-IDF, stemming, reglas) son rápidas, interpretables y no requieren GPU ni conexión a internet, pero no capturan significado profundo. Los LLM capturan semántica y generan lenguaje natural fluido, a costa de mayor costo computacional, latencia y menor transparencia ("caja negra"). El pipeline óptimo combina ambos según la restricción de cada etapa.
3. **El preprocesamiento no es neutro:** el hallazgo más relevante del laboratorio (sección 16) demuestra que una limpieza de stopwords "estándar" puede eliminar exactamente la señal que la tarea necesita (negación), y que corregirlo mejoró la accuracy de 0.58 a 0.83. Esto refuerza la importancia de validar cada etapa del pipeline con datos reales, no solo confiar en configuraciones por defecto.
4. **Aplicación directa al proyecto de curso:** el análisis de Text Mining (comentarios positivos ↔ más likes) es exactamente el tipo de insight que el proyecto "Estadísticas de YouTube" busca formalizar en un modelo predictivo, usando características lingüísticas de los comentarios como variables predictoras del desempeño del video.
5. **MLOps como requisito, no como opcional:** llevar este notebook a producción exige versionar datos y modelo, desplegar en la nube y monitorear el desempeño en el tiempo — pautas explícitamente exigidas por el curso y aplicables a cualquier proyecto real de NLP.

- Cohere LLM University. *What is Similarity Between Sentences?* Disponible en: <https://cohere.com/llmu/what-is-similarity-between-sentences>
- Cohere LLM University. *What Are Word and Sentence Embeddings?* Disponible en: <https://cohere.com/llmu/sentence-word-embeddings>
- Stanford University. *CS224N: Natural Language Processing with Deep Learning*, Lectures 12-14. Disponible en: <https://www.youtube.com/playlist?list=PLoROMvodv4rMFqRtEu06SGjY4XbRIVRd4>
- Kaggle. *YouTube Statistics Dataset*. Disponible en: <https://www.kaggle.com/datasets/advaypatil/youtube-statistics>
- Documentación oficial: NLTK (nltk.org), spaCy (spacy.io), scikit-learn (scikit-learn.org), Gensim (radimrehurek.com/gensim), Hugging Face Transformers (huggingface.co/docs/transformers).

El notebook completo, ejecutado y documentado (`Procesamiento_de_Texto_NLP.ipynb`), junto con este informe en PDF, el archivo `requirements.txt` y el código fuente, se encuentran alojados en el siguiente repositorio público de GitHub:

 **`github.com/TU-USUARIO/procesamiento-de-texto-nlp`**

Nota: reemplazar "TU-USUARIO" por el nombre de usuario real de GitHub una vez publicado el repositorio, según las instrucciones de despliegue entregadas junto con este informe.

Informe generado como parte del laboratorio de Procesamiento de Lenguaje Natural — Semanas 4 a 6. Todo el código fue ejecutado y verificado; los resultados numéricos presentados en este documento corresponden a ejecuciones reales del notebook adjunto.